our shielding · Your smart contracts, our shielding · Your smart c

shieldify

# Jailed

**Pandemic Labs**

SECURITY REVIEW

Date: 30 January 2026

# CONTENTS


**1. About Shieldify Security** **3**

**2. Disclaimer** **3**

**3. About Jailed - Pandemic Labs** **3**

**4. Risk classification** **4**

4.1 Impact 3

4.2 Likelihood 3

**5. Security Review Summary** **4**

5.1 Protocol Summary 4

5.2 Scope 4

**6. Findings Summary** **6**

**7. Findings** **8**

## 1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

## 2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

## 3. About Jailed - Pandemic Labs

A Solana-based pickaxe training and exploration game. It blends NFT-style assets, STEPN-like exploration loops, and a Token-2022 economy with JAIL. On-chain handles ownership and verification; off-chain handles computation and player experience.

### Core Mechanics

#### 1. Pickaxes

- **ID:** Unique, linked to off-chain metadata
- **HP:** Max 100, consumed during exploration
- **Stats:** Efficiency / Luck / Durability are off-chain
- **Gem slots:** 4 slots on-chain
- **Breeding count:** Max 7 times
- **Cooldown:** 24h after breeding request (starts immediately)

**Acquisition:** Admin grants, pickaxe box VRF rewards, marketplace purchases. Breeding awards pickaxe boxes (not direct pickaxes).

#### 2. Exploration

- **Loadout:** Exactly 3 main pickaxes. All three consume HP equally; equipped pickaxes cannot be listed. Gem changes are blocked only for pickaxes in the active loadout while exploring.
- **Computation:** Off-chain reward calc  on-chain claim (requires `is_exploring = true`, records HP change + nonce, then clears `is_exploring`).
- **Rewards:** On-chain claim only updates HP/nonce. JAIL/gems are settled via `open_mystery_box`; pickaxe boxes via operator grants or other flows.

Flow: set loadout  operator starts exploration  off-chain explore  operator claim  open mystery boxes / grant boxes  repair if needed.

## 4. Risk Classification

| Severity | Impact: High | Impact: Medium | Impact: Low |
|---|---|---|---|
| **Likelihood: High** | Critical | High | Medium |
| **Likelihood: Medium** | High | Medium | Low |
| **Likelihood: Low** | Medium | Low | Low |

### 4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

### 4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

## 5. Security Review Summary

The security review lasted 15 days with a total of 360 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying three Medium and ten Low severity issues. They're mainly related to state management flaws, nonce coupling, and inconsistent authorization and pause enforcement across player and marketplace workflows.

The Jailed team has done a great job and provided support and responses to all of the questions that the Shieldify researchers had.

### 5.1 Protocol Summary

| | |
|---|---|
| **Project Name** | **Jailed – Pandemic Labs** |
| **Repository** | jailed-contracts |
| **Type of Project** | GameFi, VRF |
| **Security Review Timeline** | 15 days |
| **Review Commit Hash** | 9ec79e2fcb4676ad83d1dbdd558365774d629803 |
| **Fixes Review Commit Hash** | 79d1359f3140bd1206519352479dcc7840d5c456 |

### 5.2 Scope

The following smart contracts were in the scope of the security review:

| File | Lines |
|---|---|
| programs/game/src/constants.rs | 192 |
| programs/game/src/errors.rs | 270 |
| programs/game/src/heap.rs | 14 |
| programs/game/src/lib.rs | 562 |
| programs/game/src/state.rs | 775 |
| programs/game/src/utils/instructions/mod.rs | 26 |
| programs/game/src/utils/instructions/admin/admin.rs | 812 |
| programs/game/src/utils/instructions/admin/burn.rs | 309 |
| programs/game/src/utils/instructions/admin/mod.rs | 18 |
| programs/game/src/utils/instructions/boxes/mod.rs | 12 |
| programs/game/src/utils/instructions/boxes/mystery_rewards.rs | 285 |
| programs/game/src/utils/instructions/boxes/pickaxe_boxes.rs | 369 |
| programs/game/src/utils/instructions/breeding/mod.rs | 26 |
| programs/game/src/utils/instructions/breeding/pickaxe.rs | 837 |
| programs/game/src/utils/instructions/breeding/vrf.rs | 83 |
| programs/game/src/utils/instructions/equipment/gem_inventory.rs | 71 |
| programs/game/src/utils/instructions/equipment/gem_level.rs | 420 |
| programs/game/src/utils/instructions/equipment/gems.rs | 343 |
| programs/game/src/utils/instructions/equipment/initialize.rs | 185 |
| programs/game/src/utils/instructions/equipment/mod.rs | 20 |
| programs/game/src/utils/instructions/equipment/pickaxe_inventory.rs | 67 |
| programs/game/src/utils/instructions/equipment/pickaxe_repair.rs | 205 |
| programs/game/src/utils/instructions/exploration/loadout.rs | 93 |
| programs/game/src/utils/instructions/exploration/mod.rs | 14 |
| programs/game/src/utils/instructions/exploration/rewards.rs | 307 |
| programs/game/src/utils/instructions/exploration/start.rs | 103 |

| | |
|---|---|
| programs/game/src/utils/instructions/marketplace/marketplace.rs | 931 |
| programs/game/src/utils/instructions/marketplace/mod.rs | 10 |
| programs/game/src/utils/instructions/token/jail_token.rs | 321 |
| programs/game/src/utils/instructions/token/mod.rs | 9 |
| programs/game/src/utils/domain.rs | 43 |
| programs/game/src/utils/mod.rs | 15 |
| programs/game/src/utils/shared.rs | 462 |
| programs/game/src/utils/signature.rs | 208 |
| programs/game/src/utils/vrf_helpers.rs | 68 |
| Total | 8485 |

## 6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: **3**
- **Low** issues: **10**
- **Info** issues: **2**

| ID | Title | Severity | Status |
|---|---|---|---|
| [M-01] | Nonce Exhaustion Can Permanently Brick Player Workflows (Liveness Failure) | Medium | Fixed |
| [M-02] | The `request_pickaxe_box_opening()` Removes the Box Before Randomness Is Fulfilled, Causing Permanent Data Loss Under Certain Conditions | Medium | Fixed |
| [M-03] | The `update_gems` Fails to Work for Already Equipped Gems Between Pickaxes | Medium | Fixed |
| [L-01] | Impossible to Revoke Player Access By Admin If an Operator Is Removed | Low | Fixed |
| [L-02] | Pause-flag Coupling Blocks `cancel_listing()` While `buy_item()` Can Remain Enabled | Low | Fixed |
| [L-03] | The `buy_item()` Allowlist Validation Is Asymmetric (Buyer Checked, Seller Not Checked) | Low | Acknowledged |
| [L-04] | Expiry Boundary Semantics Inconsistent Across Marketplace and Shared Helpers | Low | Fixed |
| [L-05] | Listing Requires Cooldown PDA via `remaining_accounts()` (Integration Fragility/DoS) | Low | Fixed |
| [L-06] | Gem Level-up Cost Signature Not Bound to Gem Type-/Level (Pricing Bypass If Costs Vary) | Low | Fixed |
| [L-07] | VRF Roll Not Enforced in `resolve_pickaxe_box_opening()` Breaking VRF Randomness Guarantees | Low | Acknowledged |
| [L-08] | Unrestricted Miniting of Rewards Using `open_mystery_box()` By an Operator | Low | Acknowledged |
| [L-09] | Operator Might Block Breeding Resolution | Low | Fixed |
| [L-10] | Inconsistent Pause Enforcement in `resolve_pickaxe_breeding()` Bypasses Pause Checks While Other Resolution Paths Enforce Them | Low | Fixed |
| [I-01] | Removing the Last Operator Can Brick Operator Gated Game Functions | Info | Fixed |
| [I-02] | `BreedingAttempt` Account Size Calculation Underestimates Outcome Field Size | Info | Fixed |

## 7. Findings

### [M-01] Nonce Exhaustion Can Permanently Brick Player Workflows (Liveness Failure)

#### Severity

Medium Risk

#### Description

The program uses a strictly monotonic per-player nonce ( `PlayerProfile.last_nonce` ) as replay protection across multiple signed workflows.

The only on-chain rule is `nonce > last_nonce`. There is no upper-bound check, no safety margin, and no recovery/reset path.

If the signature authority ever issues an extremely large nonce (e.g., close to `u64::MAX` ) and the player successfully executes any instruction that calls `check_and_add_nonce()` , the player's `last_nonce()` will be set to that value. From that point onward, the player may be unable to satisfy `nonce > last_nonce` for future signed actions. In the worst case (e.g., `last_nonce == u64::MAX` ), it becomes impossible to ever submit a valid higher nonce, permanently bricking the affected flows for that player.

This is a direct liveness vulnerability that can occur due to off-chain issuance bugs/misconfiguration (not necessarily an active attacker). For the marketplace specifically, `list_item()` consumes the nonce and therefore listing can become permanently unavailable for the player, leading to indirect financial loss (inability to sell/list).

#### Location of Affected Code

- Monotonic nonce rule (no upper bound / no safety margin):

File: game/src/utils/shared.rs#L410-L422

- Marketplace listing consumes the signed nonce:

File: game/src/instructions/marketplace/marketplace.rs#L431-L471

#### Impact

- Permanent or long-lived liveness failure for the affected player: future signed actions can fail with `NonceAlreadyUsed` even when signatures are otherwise valid.
- Marketplace usability failure: player can be unable to list items (and potentially other nonce-gated workflows), causing indirect economic harm.
- Operational impact: a single extreme nonce can irreversibly push a player to the edge of the `u64` space.

#### Proof of Concept

Precondition: a valid signature authority issues a signed payload using a nonce near `u64::MAX` , as an automated RANDOMIZER may not be controlled to be within bounds.

1. Obtain a valid floor signature payload where `floor_data.nonce = u64::MAX - 1` (or `u64::MAX`).
2. Execute `list_item()` successfully (or any other instruction that calls `check_and_add_nonce`).
3. Observe `PlayerProfile.last_nonce` becomes `u64::MAX - 1` (or `u64::MAX`).
4. Any later signed workflow requiring `nonce > last_nonce` becomes impossible (or has only a vanishingly small remaining nonce space), permanently preventing the player from using those workflows.

### Recommendation

- Enforce an on-chain maximum nonce with a safety margin (e.g., reject `nonce > u64::MAX - SAFETY_MARGIN`) to guarantee future headroom.
- Decouple PDA salts from replay nonces: use a bounded/structured replay counter for `last_nonce`, and use a separate unpredictable salt where needed.
- Use per-feature (per-domain) nonces instead of a single global `last_nonce` to reduce blast radius and avoid cross-feature liveness coupling.

### Team Response

Fixed.

## [M-02] The `request_pickaxe_box_opening()` Removes the Box Before Randomness Is Fulfilled, Causing Permanent Data Loss Under Certain Conditions

### Severity

Medium Risk

### Description

- In `request_pickaxe_box_opening()`, the box is irreversibly removed from `PlayerProfile.pickaxe_boxes` immediately.
- The randomness fulfilment is a **separate transaction** and can only happen when the VRF program calls:
  - `fulfill_pickaxe_box_randomness(...)`
- The reward is applied in yet another **separate transaction** by an operator:
  - `resolve_pickaxe_box_opening(...)`

So you have a 3-step async pipeline with **state burned at step 1** and value returned only at step 3.

The pickaxe box is removed from `PlayerProfile` immediately, but the VRF callback + operator resolution are separate step that can fail or never happen.

If the VRF callback never arrives or the operator never resolves, the player permanently loses the box and gets no pickaxe.

### Location of Affected Code

File: game/src/instructions/boxes/pickaxe_boxes.rs

```
/// Request to open a pickaxe box via VRF.
///
/// Burns the player's box, creates a PickaxeBoxOpenAttempt, and issues a
    MagicBlock VRF request.
/// Results are received in `fulfill_pickaxe_box_randomness` and
   finalized in `resolve_pickaxe_box_opening`.
pub fn request_pickaxe_box_opening(
    ctx: Context<RequestPickaxeBoxOpen>,
    attempt_id: u64,
    pickaxe_box_id: u32,
) -> Result<()> {
    // code

    let player_profile = &mut ctx.accounts.player_profile;
    // Locate the box in inventory
    let box_index = player_profile.pickaxe_box_index(pickaxe_box_id)?;
    // Remove (burn) the box
    player_profile.pickaxe_boxes.remove(box_index);     << This is an
       issue under certain scenarios

    // code
}
```

### Impact

- If Phase 1 succeeds (box removed + attempt created + VRF request sent), and Phase 2/3 never completes, the user is left with:
  - no box in inventory, and
  - an attempt PDA that may remain forever `AwaitingRandomness` or `RandomnessReady`.

The likelihood of the above scenarios is low under normal conditions. However, there's no on-chain cancel/refund, even rare failures can strand boxes forever.

The docs also admit this hard failure scenario – "If the VRF callback transaction is not delivered to the webhook, the resolve step cannot proceed."

Even though the docs say jobs are idempotent and deduped, which helps in retries, idempotency isn't a guarantee of eventual success.

### Proof of Concept

1. **Precondition**: Player has `PickaxeBoxItem { id = BOX_ID }` in `PlayerProfile.pickaxe_boxes`.
2. **Phase 1**: Player calls `request_pickaxe_box_opening(attempt_id, BOX_ID)`.
3. **Onchain effect**: The program **removes** that box from `player_profile.pickaxe_boxes` and creates `PickaxeBoxOpenAttempt { status = AwaitingRandomness, box_id = BOX_ID, ... }`

.

4. **liveness failure occurs**:
   - VRF callback tx never lands
   - VRF callback lands but webhook ingestion misses it (so no resolve job can be run)
5. **Wait** an arbitrary amount of time.
6. Observe on-chain:
   - `PickaxeBoxOpenAttempt.status` stays `AwaitingRandomness` or `RandomnessReady`, and
   - The original `BOX_ID` is **still missing** from `PlayerProfile.pickaxe_boxes`, and
   - There is **no instruction** in `lib.rs` that lets the player cancel the box.
7. Result: The box is effectively **stranded** (user can't complete or recover).

### Recommendation

### Option 1

Add a timeout and a player cancel instruction

- Add an instruction like `cancel_pickaxe_box_opening(attempt_id)` callable by the **player** (or anyone, but refund always goes to player), which:
  - requires `attempt.status != Resolved`
  - requires `Clock::get()?.unix_timestamp >= attempt.requested_at + TIMEOUT_SECONDS`
  - restores `PickaxeBoxItem { id: attempt.box_id }` back into `player_profile.pickaxe_boxes`
  - marks attempt as cancelled or close the attempt PDA (refund rent)

### Option 2

Escrow or lock instead of removing the box from the profile immediately.

- Mark it "locked" (like store `locked_pickaxe_box_ids` in `PlayerProfile` or store the box in the attempt and make all boxes using listing paths reject locked IDs).
- Remove only at successful resolution, and allow cancellation to unlock.

### Team Response

Fixed.

## [M-03] The `update_gems` Fails to Work for Already Equipped Gems Between Pickaxes

### Severity

Medium Risk

### Description

The `update_gems()` function is designed to allow equipping and unequipping multiple gems in a single transaction. However, it cannot move a gem that is already equipped on one pickaxe to another pickaxe in the same transaction. This limitation occurs because the function attempts to remove all gems from `player_profile.gems` inventory before performing any unequip operations.

When a gem is equipped on a pickaxe, it is removed from `player_profile.gems` and stored in the pickaxe's `equipped_gems` array. The `update_gems()` function first tries to call `player_profile.remove_gem(gem_id)` for each gem in `equip_ids`, but if a gem is already equipped, it is not present in `player_profile.gems`, causing `remove_gem()` to fail with `GameError::GemNotFound` before the unequip step can execute.

This is a functional & logical limitation as it creates a design mismatch if it's intended with user expectations for an "update (equip+unequip) in one transaction" API as per docs: \end{lstlisting} ## update_gems **Purpose**: Equip and unequip gems in one transaction - Efficiently swap equipment - Saves gas fees as a convenience function \end{lstlisting}

### Location of Affected Code

File: game/src/instructions/equipment/gems.rs#L179-L247

```rust
/// Equip and unequip multiple gems in one transaction.
pub fn update_gems(
    ctx: Context<UpdateGems>,
    pickaxe_id: u32,
    equip_ids: Vec<u32>,
    unequip_ids: Vec<u32>,
) -> Result<()> {
    // Validate allowlist + MANAGE_GEMS pause flag together
    validate_player_context!(ctx, crate::constants::pause_flags::
       MANAGE_GEMS);

    if equip_ids.is_empty() && unequip_ids.is_empty() {
        return Ok(());
    }

    let player_profile = &mut ctx.accounts.player_profile;
    let player_pickaxes = &mut ctx.accounts.player_pickaxes;
    let pickaxe_index = player_pickaxes.pickaxe_index(pickaxe_id)?;

    // Block gem changes on equipped exploration pickaxes while exploring
    if player_profile.is_exploring
        && player_profile
            .exploration_loadout
            .is_pickaxe_equipped(pickaxe_id)
    {
        return err!(GameError::ExplorationEquipmentLocked);
    }

    let equip_count = equip_ids.len();
    let unequip_count = unequip_ids.len();

    // Pre-check final inventory capacity so equip/unequip can roll back
       safely
    require!(
        player_profile.gems.len() + unequip_count >= equip_count,
        GameError::GemNotFound
    );
    require!(
        player_profile.gems.len() + unequip_count - equip_count <=
           PlayerProfile::MAX_GEMS,
        GameError::GemStorageFull
    );

    // Remove gems to be equipped first to free up inventory space
    let mut staged_gems: Vec<Gem> = Vec::with_capacity(equip_count);
    for gem_id in equip_ids {
        staged_gems.push(player_profile.remove_gem(gem_id)?);    >>>>>
           bug
    }
```

```
    // Unequip after freeing inventory slots to avoid GemStorageFull
    for gem_id in unequip_ids {
        unequip_single_gem(
            player_profile,
            player_pickaxes,
            pickaxe_index,
            pickaxe_id,
            gem_id,
        )?;
    }

    // Equip the staged gems
    for gem in staged_gems {
        equip_gem_direct(
            player_profile,
            player_pickaxes,
            pickaxe_index,
            pickaxe_id,
            gem,
        )?;
    }

    Ok(())
}
```

The issue occurs when the function attempts to remove gems from inventory before checking if they are already equipped. If a gem is currently equipped on another pickaxe, it is not in `player_profile.gems`, causing `remove_gem()` to fail with `GameError::GemNotFound` before the unequip step can execute.

### Impact

**API doesn't work as expected:** - Users cannot perform the intuitive operation of moving a gem from pickaxe A to pickaxe B in a single transaction using `update_gems()`

**Operational Impact:** - Backend must implement a 2-step flow (unequip transaction, then equip transaction), resulting in: - Increased transaction fees (two transactions instead of one) - Worse user experience (requires multiple confirmations)

**Current Workaround:** - Clients can use two separate transactions: 1. Call `unequip_gems()` on pickaxe A 2. Call `equip_gems()` on pickaxe B

However, these workarounds do not address the core limitation of the `update_gems()` function itself.

### Proof of Concept

To reproduce this issue:

1. Equip gem with ID `X` on pickaxe A

2. Attempt to call `update_gems()` with:
    - `pickaxe_id`: pickaxe B's ID
    - `equip_ids`: `[X]` (the gem currently equipped on pickaxe A)
    - `unequip_ids`: `[]` (empty, assuming the gem should be moved)

**Expected behavior:** The gem should be unequipped from pickaxe A and equipped on pickaxe B in a single transaction.

**Actual behavior:** The transaction fails with `GameError::GemNotFound` because: - function calls `player_profile.remove_gem(X)` - `remove_gem` internally calls `gem_index(X)` which searches `player_profile.gems` - Since gem X is equipped on pickaxe A, it is not in `player_profile.gems` - `gem_index` returns `GameError::GemNotFound` - The transaction reverts before the unequip step can execute

### Recommendation

Modify the `update_gems()` function to: 1. **First unequip all gems in** `unequip_ids` from the target pickaxe 2. **For each gem in** `equip_ids`**, check if it's in inventory or equipped** 3. **If equipped, find which pickaxe has it and unequip from that pickaxe** (with exploration lock check) 4. **Then remove gems from inventory and equip them** on the target pickaxe

### Team Response

Fixed.

## [L-01] Impossible to Revoke Player Access By Admin If an Operator Is Removed

### Severity

Low Risk

### Description

When an operator is removed from `config.operators`, players they granted access to cannot have their access revoked, even by the admin.

There are two conflicting checks in revoke_player_access:
1. `assert_operator_or_admin()` -> allows admin or any current operator
2. `constraint = player_access_record.granted_by == operator.key()` -> requires the signer to be the specific operator who granted access

If a compromised or removed operator previously granted access, those players become permanently allowlisted with no way to revoke their access, creating a security and business logic risk.

### Location of Affected Code

File: game/src/lib.rs

```
/// Remove a player from the allowlist
pub fn revoke_player_access(ctx: Context<RevokePlayerAccess>) -> Result
    <()> {
     // Delete the PlayerAccessRecord to revoke access
     instructions::revoke_player_access(ctx)
}
```

File: game/src/instructions/admin/admin.rs#L224-L244

```
/// Accounts for operators removing a player from the allowlist.
///
/// Closes the existing PlayerAccessRecord to return rent.
#[derive(Accounts)]
pub struct RevokePlayerAccess<'info> {
    #[account(mut)]
    pub operator: Signer<'info>,
    #[account(
        mut,
        seeds = [crate::constants::CONFIG_SEED, config.config_seed.as_ref
            ()],
        bump = config.bump
    )]
```

```

    pub config: Account<'info, GameConfig>,
    #[account(mut)]
    pub player: SystemAccount<'info>,
    #[account(
        mut,
        seeds = [crate::constants::PLAYER_ACCESS_SEED, player.key().
            as_ref()],
        bump = player_access_record.bump,
        close = operator,
        constraint = player_access_record.granted_by == operator.key() @
            GameError::InvalidAccessRevoker,  >> @bug
        constraint = player_access_record.player == player.key() @
            GameError::PlayerNotAllowlisted
    )]
    pub player_access_record: Account<'info, PlayerAccessRecord>,
}
```

File: game/src/instructions/admin/admin.rs#L463-L474

```
/// Remove a player from the allowlist (record is closed to return rent).
pub fn revoke_player_access(ctx: Context<RevokePlayerAccess>) -> Result
    <()> {
     assert_operator_or_admin(&ctx.accounts.config, &ctx.accounts.operator
        .key())?;

     let player = ctx.accounts.player.key();

     emit!(PlayerAccessRevoked {
         player,
         operator: ctx.accounts.operator.key()
     });

     Ok(())
}
```

The above constraint only allows the original grantor to revoke access. Because of this: - If Operator Bob grants access and is later removed by an admin, no one can revoke that access (not even admins).

### Impact

Admins cannot revoke access granted by operators, violating the basic admin role.

### Proof of Concept

### Initial State

- Admin: `AdminPubkey`
- Operator A: `OperatorAPubkey`
- Operator B: `OperatorBPubkey`
- Player: `PlayerPubkey`

State:
- `GameConfig.operators = [OperatorAPubkey, OperatorBPubkey]`
- `GameConfig.admin = AdminPubkey`

### Operator A Grants Access to a Player

- Operator A calls `grant_player_access()` for `PlayerPubkey`.
- A `PlayerAccessRecord` PDA is created:
  - `player = PlayerPubkey`
  - `granted_by = OperatorAPubkey`
- Player is now allowlisted.

### Player Misbehaves

- Admin/operators decide to revoke access.

### Operator A is removed by the admin

- Admin calls `remove_operator(OperatorAPubkey)`.
- `GameConfig.operators` no longer contains `OperatorAPubkey`.
- Operator A can no longer sign transactions as an operator.

### Attempt to Revoke Access by the admin

- Admin calls `revoke_player_access` for `PlayerPubkey`.
- Constraint check:

```
player_access_record.granted_by == operator.key()
```

  - `player_access_record.granted_by = OperatorAPubkey`
  - `operator.key() = AdminPubkey`
  - `OperatorAPubkey != AdminPubkey` FAIL
- Transaction fails with `InvalidAccessRevoker`.
- Admin cannot revoke access.

### Attempt to Revoke Access by operatorB

- Operator B calls `revoke_player_access()` for `PlayerPubkey`.
- Constraint check:
  - `player_access_record.granted_by = OperatorAPubkey`
  - `operator.key() = OperatorBPubkey`
  - `OperatorAPubkey != OperatorBPubkey` FAIL
- Transaction fails with `InvalidAccessRevoker`.
- Operator B cannot revoke access.

### Attempt to Re-add Operator A

- Re-adding Operator A would require:
  - Re-adding them as an operator (if they still have the keypair).
  - Having them sign the revocation.
- Problems:
  - If Operator A is malicious or unavailable, this won't work.
  - If Operator A was removed for security reasons, re-adding them is unsafe.
  - If Operator A lost their keypair, it's impossible.

### Step 8: Final State with player in Permanent Access

- Player remains allowlisted.
- No one can revoke access.
- If `enforce_player_allowlist = true`, the player can continue playing.
- If the player is banned, they retain access until allowlist enforcement is disabled.

### Recommendation

The system should allow admins to revoke any access.

- Remove the constraint from accounts struct Current code:

```
constraint = player_access_record.granted_by == operator.key() @
    GameError::InvalidAccessRevoker,
```

  Remove this constraint. Keep the other constraint that validates the player matches.

- In the `revoke_player_access()` function body, after the existing `assert_operator_or_admin()` check, add a new authorization check.
- The new check should allow revocation if the signer is the admin OR if the signer is the operator who originally granted the access.
- If neither condition is true, reject the transaction with `InvalidAccessRevoker`.
- This ensures admins can always revoke any access, while operators can only revoke access they granted themselves.

### Team Response

Fixed.

## [L-02] Pause-flag Coupling Blocks `cancel_listing()` While `buy_item()` Can Remain Enabled

### Severity

Low Risk

### Description

The `cancel_listing()` is gated by the same pause flag as `list_item()` ( `pause_flags::LIST_ITEM` ). However, purchases are gated by a different flag ( `pause_flags::BUY_ITEM` ).

This creates an operational problem: if `LIST_ITEM` is paused (commonly done to stop new listings), sellers are also prevented from cancelling existing listings, even if `BUY_ITEM` remains enabled. This can lead to a one-sided marketplace period where buyers can still purchase listings, but sellers cannot withdraw/cancel them.

### Location of Affected Code

- `buy_item()` is gated by `pause_flags::BUY_ITEM`:

File: game/src/instructions/marketplace/marketplace.rs#L532-L640

```rust
pub fn buy_item(
    ctx: Context<BuyItem>,
    item_id: u32,
    item_type: u8,
    floor_nonce: u64,
) -> Result<()> {
    validate_player_context!(ctx, buyer, crate::constants::pause_flags::
        BUY_ITEM);
    let _ = (item_id, item_type, floor_nonce);
    // code
}
```

- `cancel_listing()` is gated by `pause_flags::LIST_ITEM`:

File: game/src/instructions/marketplace/marketplace.rs#L648-L676

```rust
pub fn cancel_listing(
    ctx: Context<CancelListing>,
    item_id: u32,
    item_type: u8,
    floor_nonce: u64,
) -> Result<()> {
    validate_player_context!(ctx, seller, crate::constants::pause_flags::
        LIST_ITEM);
    let _ = (item_id, item_type, floor_nonce);
    // code
}
```

### Impact

- During a `LIST_ITEM` pause window, sellers cannot cancel active listings.
- If `BUY_ITEM` remains enabled, active listings can still be purchased, but sellers cannot react by withdrawing them.
- This is a liveness/policy discrepancy that can cause user harm under normal operational pausing.

### Proof of Concept

1. Create a listing successfully.
2. Pause `LIST_ITEM` while keeping `BUY_ITEM` unpaused.
3. Attempt to cancel the listing; it fails due to pause gating.
4. A buyer can still purchase the listing because `BUY_ITEM` remains enabled.

### Recommendation

- Introduce a dedicated pause flag for cancellation (e.g., `CANCEL_LISTING`) and gate `cancel_listing()` on it.
- Alternatively, gate `cancel_listing()` using `BUY_ITEM` (or allow cancel when either flag is unpaused), depending on intended operational policy.

### Team Response

Fixed.

## [L-03] The `buy_item()` Allowlist Validation Is Asymmetric (Buyer Checked, Seller Not Checked)

### Severity

Low Risk

### Description

The allowlist + pause validation in `buy_item()` is applied only to the `buyer` via `validate_player_context!(ctx, buyer, BUY_ITEM)`. The seller is not allowlist-validated in the buy path.

In contrast, both `list_item()` and `cancel_listing()` validate the seller with `validate_player_context!(ctx, seller, LIST_ITEM)`.

This creates a policy inconsistency: a seller who is no longer allowlisted (or whose allowlist enforcement status changes) may be prevented from listing/cancelling, while their existing listings can still be purchased by allowlisted buyers.

### Location of Affected Code

- Buyer allowlist validation in `buy_item()`:

File: game/src/instructions/marketplace/marketplace.rs#L539

```
pub fn buy_item(
    ctx: Context<BuyItem>,
    item_id: u32,
    item_type: u8,
    floor_nonce: u64,
) -> Result<()> {
    validate_player_context!(ctx, buyer, crate::constants::pause_flags::
        BUY_ITEM);
    // code
}
```

- Seller allowlist validation in `list_item()`:

File: game/src/instructions/marketplace/marketplace.rs#L452

```rust
pub fn list_item<'info>(
    ctx: Context<'_, '_, '_, 'info, ListItem<'info>>,
    item_id: u32,
    price: u64,
    item_type: u8,
    floor_nonce: u64,
    floor_data: FloorListingSignatureData,
    floor_signature: [u8; 64],
) -> Result<()> {
    // code
    validate_player_context!(ctx, seller, crate::constants::pause_flags::
        LIST_ITEM);
    // code
}
```

- Seller allowlist validation in `cancel_listing()`:

File:game/src/instructions/marketplace/marketplace.rs#L655

```rust
pub fn cancel_listing(
    ctx: Context<CancelListing>,
    item_id: u32,
    item_type: u8,
    floor_nonce: u64,
) -> Result<()> {
    validate_player_context!(ctx, seller, crate::constants::pause_flags::
        LIST_ITEM);
    // code
}
```

### Impact

- Policy mismatch under allowlist enforcement:
  - Sellers can be blocked from managing their own listings (list/cancel),
  - while buyers can still purchase those listings.
- Creates user-facing liveness surprises during allowlist toggles or access revocations.

### Proof of Concept

1. Seller creates a listing successfully while allowlisted.
2. Seller later becomes not-allowlisted (or allowlist enforcement toggles).
3. Seller attempts `cancel_listing()` and fails due to seller allowlist validation.
4. An allowlisted buyer executes `buy_item()` successfully because only the buyer is checked.

### Recommendation

Decide the intended policy: - If the allowlist should govern all marketplace participation, also validate the seller in `buy_item()`. - If allowlist should only gate active actors, document this explicitly and consider allowing sellers to always cancel their own listings (even when not allowlisted) to reduce liveness harm.

### Team Response

Acknowledged.

## [L-04] Expiry Boundary Semantics Inconsistent Across Marketplace and Shared Helpers

### Severity

Low Risk

### Description

Marketplace listing floor signature expiry uses an inclusive comparison ( `now <= expires_at` ), while other parts of the codebase use strict comparisons via the shared helper `validate_timestamp()` (which requires `current_time < timestamp` ).

This inconsistency creates hard-to-debug boundary failures (exact-second mismatches) and can cause off-chain systems to behave unpredictably when issuing signatures close to expiry.

### Location of Affected Code

File: game/src/instructions/marketplace/marketplace.rs#L456-L461

```
pub fn list_item<'info>(
    ctx: Context<'_, '_, '_, 'info, ListItem<'info>>,
    item_id: u32,
    price: u64,
    item_type: u8,
    floor_nonce: u64,
    floor_data: FloorListingSignatureData,
    floor_signature: [u8; 64],
) -> Result<()> {
    // code
    let clock = Clock::get()?;
    require!(
        clock.unix_timestamp <= floor_data.expires_at,
        GameError::FloorSignatureExpired
    );
    // code
}
```

File: game/src/utils/shared.rs#L456-L460

```
pub fn validate_timestamp(timestamp: i64, current_time: i64) -> Result<()
   > {
    require!(current_time < timestamp, GameError::RewardExpired);
    Ok(())
}
```

### Impact

- Off-by-one-second failures near expiry.
- Increased false negatives in signature validation flows.

### Recommendation

- Standardize expiry semantics across the program (choose either inclusive or strict) and apply consistently.
- Align shared helper naming/docs with its actual comparison to reduce future misuse.

### Team Response

Fixed.

## [L-05] Listing Requires Cooldown PDA via `remaining_accounts()` (Integration Fragility/DoS)

### Severity

Low Risk

### Description

Pickaxe listings require a `PickaxeBreedingCooldown` PDA to be passed via `ctx.remaining_accounts`. If the account is omitted, listing fails with `MissingPickaxeBreedingCooldownAccount`.

Although not a fund-loss issue, this increases transaction fragility and makes CPI/integration correctness harder. It also creates an easy self-DoS for clients that build account metas incorrectly.

### Location of Affected Code

File: game/src/instructions/marketplace/marketplace.rs#L778-L782

```
fn ensure_pickaxe_not_on_cooldown<'info>(
    remaining_accounts: &[AccountInfo<'info>],
    payer: &AccountInfo<'info>,
    system_program: &AccountInfo<'info>,
    owner: &Pubkey,
    pickaxe_id: u32,
    now: i64,
) -> Result<()> {
    // code
    let (expected_pda, _bump) = Pubkey::find_program_address(&seeds, &
       crate::ID);
    let cooldown_info = remaining_accounts
        .iter()
        .find(|account| account.key() == expected_pda)
        .ok_or(GameError::MissingPickaxeBreedingCooldownAccount)?;
    // code
}
```

### Impact

- Higher failure rate for valid user actions due to missing metas.
- CPI integrators must replicate PDA derivations and always include the account.

### Proof of Concept

Submit `list_item()` for `MarketplaceItemType::Pickaxe` without including the cooldown PDA account in remaining accounts, and observe failure.

### Recommendation

- Consider moving the cooldown PDA into the explicit `Accounts` struct (with seed constraints) to eliminate reliance on remaining accounts.
- If remaining accounts are required, provide strict client-side derivation utilities and enforce ordering/dup checks.

### Team Response

Fixed.

## [L-06] Gem Level-up Cost Signature Not Bound to Gem Type/Level (Pricing Bypass If Costs Vary)

### Severity

Low Risk

### Description

The `request_gem_level_up()` consumes three gems (validated to be the same type and level) and then verifies an admin-signed `BreedingCostSignature` before burning the signed JAIL cost.

However, the signed payload used for gem level-ups does not commit to the gem type, base level, or the consumed gem IDs. The on-chain signature message for `BreedingAttemptType::Gem` only includes `(attempt_id, player, cost, nonce, expires_at)`.

If the backend intends to price gem level-ups differently based on gem type and/or base level, a player can present a valid signature for a cheaper scenario while consuming higher-value gems on-chain, paying the cheaper signed cost.

### Location of Affected Code

- Gem consumption happens before burning the signed cost:

File: game/src/instructions/equipment/gem_level.rs#L78-L99

```rust
pub fn request_gem_level_up(
    ctx: Context<RequestGemLevelUp>,
    attempt_id: u64,
    gem_ids: [u32; GEM_CONSUME_COUNT],
    cost_data: BreedingCostSignature,
    cost_signature: [u8; 64],
) -> Result<()> {
    // code
    // Consume three gems of the same type/level and capture their type/
       level
    let (gem_type, gem_level) = prepare_gem_consumption(player_profile,
       gem_ids)?;

    // Verify Ed25519 signature on the cost payload
    verify_cost_signature(
        &cost_data,
        attempt_id,
        BreedingAttemptType::Gem,
        player_key,
        &cost_signature,
        ctx.accounts.game_config.signature_authority,
        &ctx.accounts.instructions_sysvar,
    )?;

    // Register nonce per signature to prevent replay
    check_and_add_nonce(player_profile, cost_data.nonce)?;
    burn_jail_if_needed(

        cost_data.cost,
    )?;

    // code
}
```

- Gem cost signature message does not include gem type/level/IDs:

File: game/src/instructions/breeding/pickaxe.rs#L671-L678

```rust
fn build_signature_message(cost_data: &BreedingCostSignature) -> String {
    let message_body = match cost_data.attempt_type {
        // Gem breeding signature format
        BreedingAttemptType::Gem => format!(
            "breed_gem:{}:{}:{}:{}:{}",
            cost_data.attempt_id,
            cost_data.player,
            cost_data.cost,
            cost_data.nonce,
            cost_data.expires_at
        ),
        // code
}
```

### Impact

- If gem level-up costs are intended to vary by gem type/level, a player can pay an unintended (lower) JAIL cost for higher-value upgrades.
- If costs are intentionally constant across all gem level-ups, the current design is consistent and this becomes a documentation/assumption risk rather than a pricing bypass.

### Proof of Concept

Applicable when the off-chain pricing model varies by gem type or base level:

1. Obtain a valid `BreedingCostSignature` for a low-cost gem level-up (e.g., low base level), signed with some `cost_data.cost = X`.
2. Submit `request_gem_level_up()` consuming three gems of a higher base level (still the same type/level among the three), while reusing the signature from step 1.
3. Observe that the instruction succeeds and burns `X` JAIL, even though the higher base level would normally require a higher cost.

### Recommendation

- Consider binding the signed cost message to the on-chain consumables by including at least `gem_type` and `base_level` in the signed payload.
- For stronger binding, also include the `gem_ids` array (or a hash of it) to prevent signature reuse across different consumed sets.
- Ensure the on-chain verifier checks that the additional fields (type/level/IDs) match the consumed gems before burning the cost.

### Team Response

Fixed.

## [L-07] VRF Roll Not Enforced in `resolve_pickaxe_box_opening()` Breaking VRF Randomness Guarantees

### Severity

Low Risk

### Description

- The VRF callback writes `attempt.roll` into `PickaxeBoxOpenAttempt` ( `fulfill_pickaxe_box_randomness()` sets `attempt.roll = Some(roll_value)` and `status = RandomnessReady` ).
- But in `resolve_pickaxe_box_opening()`, the program :
  - **only checks** the attempt is `RandomnessReady`,
  - **does not compute** an expected reward from `attempt.roll`,

- **does not check** that `reward.pickaxe_id` is consistent with the roll,
- then **mints whatever pickaxe ID the operator supplies** into `PlayerPickaxeInventory`.

From the docs in the function.md VRF breeding functions – 1. Operator determines reward box IDs from rolls 2. Twin chance is off-chain

This means that – `resolve_pickaxe_breeding()` accepts `awarded_box_ids` from the operator and does not enforce that: – The number of boxes (1 vs 2) matches `secondary_roll` or – Any rarity/twin thresholds are satisfied and – The specific box IDs correspond to roll-derived tiers.

So the contract is not VRF-enforced rewards, it is VRF that provides rolls, and the operator decides outcomes.

Even though it could be argued that the operator and backend are trusted, this is a fairness vulnerability where the on-chain does not enforce success based on roll, the operator supplies success.

If the goal is provably fair, trust minimized breeding outcomes. Operator/backend can compromise, inflate or deny rewards.

### Location of Affected Code

File: game/src/instructions/boxes/pickaxe_boxes.rs#L241-L313

```
pub fn resolve_pickaxe_box_opening<'info>(
    ctx: Context<'_, '_, '_, 'info, ResolvePickaxeBoxOpen<'info>>,
    attempt_id: u64,
    rewards: PickaxeBoxRewards,
) -> Result<()> {
    instructions::resolve_pickaxe_box_opening(ctx, attempt_id, rewards)
}
```

File: game/src/instructions/boxes/pickaxe_boxes.rs#L196-L201

```rust
/// Accounts used when finalizing rewards from VRF randomness
#[derive(Accounts)]
#[instruction(attempt_id: u64, rewards: PickaxeBoxRewards)]
pub struct ResolvePickaxeBoxOpen<'info> {
    #[account(
        mut,
        constraint = game_config.operators.contains(&operator.key()) @
            GameError::UnauthorizedOperator
    )]
    pub operator: Signer<'info>,

    #[account(mut, address = box_attempt.player)]
    pub player: SystemAccount<'info>,

    #[account(
        mut,
        close = player,
        seeds = [PICKAXE_BOX_ATTEMPT_SEED, box_attempt.player.as_ref(), &
            attempt_id.to_le_bytes()],
        bump = box_attempt.bump
    )]
    pub box_attempt: Account<'info, PickaxeBoxOpenAttempt>,

    #[account(
        mut,
        seeds = [crate::constants::PLAYER_PROFILE_SEED, box_attempt.
            player.as_ref()],
        bump = player_profile.bump,
        constraint = player_profile.owner == box_attempt.player
    )]
    pub player_profile: Account<'info, PlayerProfile>,
    #[account(
        mut,
        seeds = [crate::constants::PLAYER_PICKAXES_SEED, box_attempt.
            player.as_ref()],
        bump = player_pickaxes.bump,
    )]
    pub player_pickaxes: Account<'info, PlayerPickaxeInventory>,

    #[account(
        seeds = [constants::CONFIG_SEED, game_config.config_seed.as_ref()
            ],
        bump = game_config.bump
    )]
    pub game_config: Account<'info, GameConfig>,

    pub system_program: Program<'info, System>,
}
```

File: game/src/instructions/boxes/pickaxe_boxes.rs#L241-L313

```rust
/// Finalize a pickaxe box opening after VRF is ready.
///
/// Validates operator auth, attempt state, nonce, capacity, and
   duplicate IDs before adding the
/// new pickaxes and marking the attempt as resolved.
pub fn resolve_pickaxe_box_opening<'info>(
    ctx: Context<'_, '_, '_, 'info, ResolvePickaxeBoxOpen<'info>>,
    attempt_id: u64,
    rewards: PickaxeBoxRewards,
) -> Result<()> {
    // Check allowlist + OPEN_PICKAXE_BOX pause (same gate as request)
    validate_player_context!(
        ctx.accounts.game_config,
        ctx.accounts.player_profile.owner,
        ctx.remaining_accounts,
        crate::constants::pause_flags::OPEN_PICKAXE_BOX
    );

    let attempt = &mut ctx.accounts.box_attempt;
    ensure_box_attempt_pda(attempt)?;
    require!(
        attempt.attempt_id == attempt_id,
        GameError::InvalidRandomnessRequest
    );
    require!(
        attempt.status == RandomnessStatus::RandomnessReady,
        GameError::RandomnessRequestNotReady
    );
    if attempt.resolved_at.is_some() {
        return err!(GameError::RandomnessRequestAlreadyResolved);
    }
    require!(
        rewards.pickaxes.len() == 1,
        GameError::InvalidRewardBatch
    );
```

```
    ensure_profile_capacity(
        &mut ctx.accounts.player_profile,
        &ctx.accounts.operator,
        &ctx.accounts.system_program,
        CapacityRequest::pickaxe_boxes(0).with_gems(0),
    )?;

    check_and_add_nonce(&mut ctx.accounts.player_profile, rewards.nonce)
       ?; // @audit-issue Nonce is not bound to the attempt (can be "any
       next "nonce)

    let incoming_pickaxe_count = rewards.pickaxes.len();
    check_pickaxe_inventory_limit(&ctx.accounts.player_pickaxes,
       incoming_pickaxe_count)?;

    let operator_info = ctx.accounts.operator.to_account_info();
    let system_program_info = ctx.accounts.system_program.to_account_info
       ();
    for reward in rewards.pickaxes.iter() {
        register_item_id(
            ctx.remaining_accounts,
            &operator_info,
            &system_program_info,
            reward.pickaxe_id,
            GameError::PickaxeIdAlreadyExists,
        )?;
        // Initialize every pickaxe with default stats to avoid trusting
           external data
        let properly_initialized_pickaxe = ExplorationPickaxe::new(reward
           .pickaxe_id)?;
        ctx.accounts
            .player_pickaxes
            .pickaxes
            .push(properly_initialized_pickaxe);
    }

    finalize_box_attempt(attempt, ctx.accounts.operator.key())?;

    emit!(PickaxeBoxOpenResolved {
        player: attempt.player,
        attempt_id,
        box_id: attempt.box_id,
        roll: attempt.roll.unwrap_or(0),
        pickaxe_count: rewards.pickaxes.len() as u8,
    });

    Ok(())
}
```

### Impact

The program requests VRF randomness and stores a roll, but the roll does not constrain the reward.

This means that => Any authorized operator can **mint any pickaxe ID** which is subject only to "ID unused" and storage capacity, thus enabling: – Operator can always award a `common` pickaxe for high rolls, i.e. denial of rare drops. – Always resolve with rare IDs for selected players/accounts, which could inflate the entire game economy and make it a biased game.

This will ultimately break the provably fair VRF-based randomness guarantee.

### Proof of Concept

1. **Player creates VRF attempt** – Have the player call `request_pickaxe_box_opening(attempt_id, pickaxe_box_id)` to create the `PickaxeBoxOpenAttempt` PDA and request randomness.
2. **Wait for VRF callback** – Let `fulfill_pickaxe_box_randomness(randomness)` run so that `box_attempt.status == RandomnessReady` and `box_attempt.roll == Some(roll)`.
3. **Choose an arbitrary reward** – As the operator, pick any fresh `pickaxe_id` you wish to grant (could be an ID corresponding to a "rare" off-chain metadata entry), ensuring it's unused so `register_item_id()` won't fail.
4. **Pick a valid nonce** – Fetch `player_profile.last_nonce` and set `rewards.nonce = last_nonce + 1` or any larger value.
5. **Resolve with mismatched reward** – Submit `resolve_pickaxe_box_opening(attempt_id, rewards)` where `rewards.pickaxes = [{ pickaxe_id: YOUR_CHOSEN_ID }]`, signed by the operator.
6. **On-chain result** – The program will succeed because it only checks `attempt status + nonce monotonicity + ID uniqueness & capacity` and it will mint the operator-chosen pickaxe into `PlayerPickaxeInventory` without validating against `box_attempt.roll`.

Now we get a situation where you have `PlayerPickaxeInventory` with the minted pickaxe that has `id = YOUR_CHOSEN_ID`, while the emitted event still logs the VRF roll (showing the roll existed but didn't constrain the reward).

So the `reward.pickaxe_id` is not consistent with the attempt. roll, or it belongs to a specific role derived from rarity.

### Recommendation

Implement an on-chain reward rule table and derive the reward deterministically from `attempt.roll` instead of letting an operator derive the success.

### Team Response

Acknowledged.

## [L-08] Unrestricted Miniting of Rewards Using `open_mystery_box()` By an Operator

### Severity

Low Risk

### Description

The instruction never proves the player earned a mystery box. There is no on-chain box balance, no linkage to `claim_exploration_rewards` & no signature from a separate authority. It only checks that the caller is an operator + nonce monotonicity, then mints JAIL & appends gems. There is no `mystery_boxes` counter in the system. So the program has no state to prove `a box was earned` or to prevent opening more boxes than earned.

- **There is no on-chain "mystery box inventory" to decrement or burn.**

The instruction only: – checks operator is in `GameConfig.operators` – checks pause/allowlist – calls `check_and_add_nonce` (monotonic, but an operator can always choose a larger nonce) – **mints JAIL** and **pushes a gem** into the profile.

### Location of Affected Code

File: game/src/lib.rs#L346-L352

```
/// Open mystery boxes obtained from exploration (operator-signed rewards
    ).
pub fn open_mystery_box<'info>(
    ctx: Context<'_, '_, '_, 'info, OpenMysteryBox<'info>>,
    reward_batch: MysteryBoxRewardBatch,
) -> Result<()> {
    // Validate via operator signature and distribute rewards (JAIL/gems)
    instructions::open_mystery_box(ctx, reward_batch)
}
```

File: game/src/instructions/boxes/mystery_rewards.rs#L103-L135

```rust
/// Execute mystery box openings.
/// Applies operator-provided rewards (up to 10 boxes), mints JAIL, adds
   gems, and tracks
/// nonces to prevent replay.
pub fn open_mystery_box<'info>(
    ctx: Context<'_, '_, '_, 'info, OpenMysteryBox<'info>>,
    reward_batch: MysteryBoxRewardBatch,
) -> Result<()> {
    // Enforce pause/allowlist gates for opening mystery boxes
    validate_player_context!(ctx, crate::constants::pause_flags::
       OPEN_MYSTERY_BOX);

    let rewards = reward_batch.rewards;
    validate_reward_batch(&rewards, ctx.accounts.game_config.
       max_mystery_box_jail)?;
    let total_new_gems = aggregate_reward_totals(&rewards);

    ensure_profile_capacity(
        &mut ctx.accounts.player_profile,
        &ctx.accounts.authority,
        &ctx.accounts.system_program,
        CapacityRequest::pickaxe_boxes(0)
            .with_gems(total_new_gems),
    )?;

    check_mystery_box_storage_limits(&ctx.accounts.player_profile,
       total_new_gems)?;

    // Apply each box reward in the batch
    for reward in rewards {
        apply_mystery_box_reward(
            ctx.accounts,
            reward,
            ctx.remaining_accounts,
            ctx.bumps.jail_mint_authority,
        )?;
    }

    Ok(())
}
```

File: game/src/instructions/boxes/mystery_rewards.rs#L210-L263

```rust
/// Apply a single mystery box reward, minting JAIL and adding gems.
fn apply_mystery_box_reward<'info>(
    accounts: &mut OpenMysteryBox<'info>,
    reward: MysteryBoxReward,
    remaining_accounts: &[AccountInfo<'info>],
    mint_authority_bump: u8,
) -> Result<()> {
    let MysteryBoxReward {
        box_id,
        jail_amount,
        gems,
        nonce,
    } = reward;
    let gem_ids: Vec<u32> = gems.iter().map(|g| g.id).collect();
    // Prevent replay by recording the nonce
    {
        let player_profile = &mut accounts.player_profile;
        check_and_add_nonce(player_profile, nonce)?;
    }

    // Mint JAIL only when requested
    if jail_amount > 0 {
        mint_jail_to_player(
            &accounts.jail_mint,
            &accounts.jail_mint_authority,
            &accounts.player_jail_account,
            &accounts.token_program,
            jail_amount,
            mint_authority_bump,
        )?;
    }
```

```
    let authority_info = accounts.authority.to_account_info();
    let system_program_info = accounts.system_program.to_account_info();
    {
        let player_profile = &mut accounts.player_profile;
        add_reward_gems(
            player_profile,
            gems,
            remaining_accounts,
            &authority_info,
            &system_program_info,
        )?;
    }
    emit!(MysteryBoxOpened {
        player: accounts.player.key(),
        operator: accounts.authority.key(),
        box_id,
        jail_amount,
        gem_ids,
        nonce,
    });

    Ok(())
}
```

### Impact

Any operator can repeatedly call `open_mystery_box()` for any player and mint up to `max_mystery_box_jail` each time and inject gems, limited only by operator key control.

### Proof of Concept

1. Operator constructs `MysteryBoxRewardBatch` with 10 rewards. Each reward has a `unique box_id`, `jail_amount = max_mystery_box_jail`, `gems = []`, and a monotonically increasing `nonce`.
2. Attacker submits a transaction calling `open_mystery_box(player, reward_batch)` signed by the operator key (and includes any required `ItemIdRegistry` accounts if minting gems).
3. Program accepts because the operator is authorized and each nonce > last_nonce.
4. Program mints JAIL directly into the player's token account (or appends gems) without proving any box was earned.
5. Operator repeats the transaction with higher nonces to mint again indefinitely.

### Recommendation

Add an on-chain "mystery box balance" and require burning or decrementing it on `open_mystery_box()`
.

### Team Response

Acknowledged.

## [L-09] Operator Might Block Breeding Resolution

### Severity

Low Risk

### Description

The `resolve_pickaxe_breeding()` handler is used by an operator to finalize a previously requested pickaxe breeding attempt once VRF randomness is available. At request time, a player submits a signed off-chain cost quote that includes an `expires_at` timestamp, which is intended to bound the validity of the quote and prevent replay. The program records the breeding attempt, burns the player's tokens, and requests VRF randomness, after which resolution is expected to occur asynchronously.

However, during resolution, the handler enforces `now <= breeding_attempt.expires_at`, tying the quote's expiry to the resolution timestamp rather than to the time at which randomness was fulfilled. Because VRF fulfilment and operator resolution are separate stages that may be delayed, a valid attempt can become unresolvable if resolution occurs after expiry, even when randomness was correctly produced while the quote was still valid.

### Location of Affected Code

File: game/src/instructions/breeding/pickaxe.rs#L334

```
pub fn resolve_pickaxe_breeding<'info>(...) -> Result<()> {
    let breeding_attempt = &mut ctx.accounts.breeding_attempt; //
       Validate attempt ID/type matches
    ensure_attempt_matches(breeding_attempt, attempt_id,
       BreedingAttemptType::Pickaxe)?;
    // Confirm VRF rolls are ready
    require!(
        breeding_attempt.status == RandomnessStatus::RandomnessReady,
        GameError::RandomnessRequestNotReady
    );
    // Re-check signature expiry to block late resolution
    let now = Clock::get()?.unix_timestamp;
>>  require!(now <= breeding_attempt.expires_at, GameError::RewardExpired
   );
    if breeding_attempt.outcome.is_some() {
        // Prevent re-execution if already resolved
        return err!(GameError::RandomnessRequestAlreadyResolved);
    }

    // code
```

### Impact

A legitimate breeding attempt can become permanently stuck, leaving the player without rewards after having already paid the breeding cost. Players may lose burned JAIL tokens with no box delivery, attempt accounts may remain pending and consume on-chain resources, and downstream game logic, depending on successful resolution, can be blocked. This breaks economic invariants, degrades availability, and undermines player trust in paid game actions.

### Proof of Concept

1. Bob obtains a signed breeding cost quote with `expires_at = 1000` and submits `request_pickaxe_breeding()` at time 990.
2. The program accepts the request, burns Bob's tokens, and requests VRF randomness.
3. Due to oracle or network latency, VRF randomness is fulfilled at time 995, but the operator does not call `resolve_pickaxe_breeding()` until time 1005.
4. When the operator attempts resolution, the handler evaluates `require!(now <= breeding_attempt.expires_at, GameError::RewardExpired)` and fails because `now = 1005`.
5. The breeding attempt cannot be resolved despite valid randomness, and Bob receives no rewards for the cost he already paid.

### Recommendation

Decouple signature expiry from resolution timing by enforcing that VRF randomness was fulfilled before the quote expired, rather than requiring resolution to occur before expiry.

### Team Response

Fixed.

## [L-10] Inconsistent Pause Enforcement in `resolve_pickaxe_breeding()` Bypasses Pause Checks While Other Resolution Paths Enforce Them

### Severity

Low Risk

### Description

The `resolve_pickaxe_breeding()` function does not check pause flags, unlike other operator resolution paths ( `resolve_gem_level_up()` , `resolve_pickaxe_box_opening()` , `claim_exploration_rewards()` , `open_mystery_box()` ), which do.

This creates an inconsistency:
- `request_pickaxe_breeding()` checks pause via `validate_breeding_access()`
- `resolve_pickaxe_breeding()` only checks operator membership and does not check pause

During an emergency pause, operators can still resolve breeding attempts and award pickaxe boxes, which undermines the pause mechanism.

### Location of Affected Code

File: game/src/instructions/breeding/pickaxe.rs#L320-L409

```rust
/// Resolve pickaxe breeding and award pickaxe boxes.
///
/// Operator confirms VRF roll + nonce before finalizing reward box IDs.
pub fn resolve_pickaxe_breeding<'info>(
    ctx: Context<'_, '_, '_, 'info, ResolvePickaxeBreeding<'info>>,
    attempt_id: u64,
    awarded_box_ids: Vec<u32>,
) -> Result<()> {
    let breeding_attempt = &mut ctx.accounts.breeding_attempt;
    ensure_attempt_matches(breeding_attempt, attempt_id,
        BreedingAttemptType::Pickaxe)?;
    // ... no pause check here ...

    // Only operator membership is checked:
    // constraint = game_config.operators.contains(&operator.key())
}
```

**Comparison with other resolution paths**:

- `resolve_gem_level_up()` checks `BURN_GEMS_FOR_ALCHEMY` pause flag
- `resolve_pickaxe_box_opening()` checks `OPEN_PICKAXE_BOX` pause flag
- `claim_exploration_rewards()` checks `CLAIM_EXPLORATION_REWARDS` pause flag
- `open_mystery_box()` checks `OPEN_MYSTERY_BOX` pause flag

### Impact

- Incident response bypass: during emergency pause, operators can still resolve breeding attempts and award rewards, contradicting the pause intent.
- Inconsistent behaviour: different resolution paths handle pause differently, creating confusion and potential policy violations.

### Proof of Concept

1. Admin calls `emergency_pause_all()` or `toggle_function_pause(BREED_PICKAXE, true)`.
2. Player requests breeding: `request_pickaxe_breeding()` correctly fails due to pause check.
3. VRF fulfils randomness: breeding attempt reaches `RandomnessReady`.
4. Operator resolves during pause: `resolve_pickaxe_breeding()` succeeds because it does not check pause, allowing rewards to be awarded despite the pause.

Expected behaviour: `resolve_pickaxe_breeding()` should fail with `FunctionPaused` or `GloballyPaused`, consistent with other resolution paths.

### Recommendation

Add pause check to `resolve_pickaxe_breeding()` for consistency:

```rust
pub fn resolve_pickaxe_breeding<'info>(
    ctx: Context<'_, '_, '_, 'info, ResolvePickaxeBreeding<'info>>,
    attempt_id: u64,
    awarded_box_ids: Vec<u32>,
) -> Result<()> {
    // Add pause check for consistency with other resolution paths
    crate::require_not_paused!(
        ctx.accounts.game_config,
        crate::constants::pause_flags::BREED_PICKAXE
    );

    let breeding_attempt = &mut ctx.accounts.breeding_attempt;
    ensure_attempt_matches(breeding_attempt, attempt_id,
       BreedingAttemptType::Pickaxe)?;
    // code
}
```

### Team Response

Fixed.

## [I-01] Removing the Last Operator Can Brick Operator Gated Game Functions

### Severity

Informational Risk

### Description

The `remove_operator()` admin instruction allows the admin to remove operators until the `GameConfig.operators` list becomes empty.

`There exists no minimum operator requirement.`

Many core game actions are **operator-gated**, so an empty operator set makes these flows impossible to execute.

This can cause a full availability outage until an operator is re-added, and can easily happen during routine operator rotation or by accidental removal.

While recoverable, i.e. admin can add an operator back, it creates avoidable downtime and incident-response risk. This is an operational risk.

### Location of Affected Code

File: game/src/instructions/admin/admin.rs#L797-L812

```rust
/// Remove an operator.
/// Requires admin authority and ensures the operator exists before
   removal.
pub fn remove_operator(ctx: Context<ManageOperator>, operator: Pubkey) ->
    Result<()> {
    let config = &mut ctx.accounts.config;

    let position = config.operators.iter().position(|&x| x == operator);
    require!(position.is_some(), GameError::OperatorNotFound);

    config.operators.remove(position.unwrap()); // @audit [M-1]

    emit!(OperatorRemoved {
        admin: ctx.accounts.admin.key(),
        operator,
        remaining_operators: config.operators.len() as u8,
    });

    Ok(())
}
```

### Impact

- `start_exploration()` requires an operator
- `claim_exploration_rewards()` requires an operator
- `open_mystery_box()` requires an operator
- `resolve_pickaxe_breeding()` requires an operator
- `resolve_gem_level_up()` requires an operator
- `resolve_pickaxe_box_opening()` requires an operator

If all operators are removed, these functions become unusable until an operator is added back.

### Proof of Concept

1. **Initial state**: `GameConfig.operators = [Op1]` (only one operator exists) and the game has pending/normal operator-required actions (exploration settle, VRF resolves, etc.).
2. **Admin action**: Admin calls `remove_operator(Op1)`; the transaction succeeds and now `GameConfig.operators = []`.
3. **Trigger failure**: Any operator-gated instruction is attempted (like `start_exploration()`, `claim_exploration_rewards()`, `open_mystery_box()`, `resolve_pickaxe_breeding()` etc).
4. **Observed behaviour**: The instruction fails at the operator constraint check ( `game_config.operators.contains(&authority.key())` ), because the operators list is empty.

5. **Impact**: The system is effectively "operator-deadlocked" untilthe admin adds a new operator back via `add_operator(new_op)`.
6. During the deadlock, async pipelines can remain stuck as `RandomnessReady` but not resolved.

### Recommendation

Add a minimum operator requirement, or at least one, so that the system doesn't halt:

```
pub fn remove_operator(ctx: Context<ManageOperator>, operator: Pubkey) ->
    Result<()> {
    let config = &mut ctx.accounts.config;

    let position = config.operators.iter().position(|&x| x == operator);
    require!(position.is_some(), GameError::OperatorNotFound);

    // Prevent removing the last operator (system needs at least one)
    require!(
        config.operators.len() > 1,
        GameError::CannotRemoveLastOperator  // Add this error variant
    );

    config.operators.remove(position.unwrap());
    // code
}
```

### Team Response

Fixed.

## [I-02] `BreedingAttempt` Account Size Calculation Underestimates Outcome Field Size

### Severity

Informational Risk

### Description

The `BreedingAttempt` account size calculation underestimates the serialized size of `outcome: Option<BreedingOutcome>`. While this calculation is technically incorrect, it is **not exploitable in the current implementation** due to buffer space from gem capacity allocation. However, the calculation should be corrected for code correctness and future-proofing.

`BreedingAttempt` underestimates the serialized size of `outcome: Option<BreedingOutcome>`. This should include **Option tag + enum variant tag + max variant payload**. The current constant only accounts for `1 + PickaxeBreedingOutcome::SERIALIZED_SIZE`, which is too small.

```rust
impl BreedingAttempt {
    pub const MAX_TRACKED_GEMS: usize = 3;
    pub const MAX_AWARDED_BOXES: usize = 2;
    const OUTCOME_MAX_SIZE: usize = 1 + PickaxeBreedingOutcome::
       SERIALIZED_SIZE;   >>>>>>   @bug

    const BASE_SIZE: usize = 8  // discriminator
        // code
        + Self::OUTCOME_MAX_SIZE // outcome data (max pickaxe variant)
        // code
        + 1; // bump

    pub fn space(consumed_gems: usize) -> usize {
        let gem_capacity = core::cmp::max(consumed_gems, Self::
           MAX_TRACKED_GEMS);
        Self::BASE_SIZE + gem_capacity * 4
    }
}

impl PickaxeBreedingOutcome {
    pub const SERIALIZED_SIZE: usize = 1 /* twin_triggered */
        + 1 /* boxes_awarded */
        + 2 /* rarity_roll */
        + 2; /* twin_roll */
}
```

```rust
pub struct GemBreedingOutcome {
    pub success: bool,            // 1
    pub new_gem_id: Option<u32>,  // 1 + 4 = 5
    pub new_gem_level: Option<u8> // 1 + 1 = 2
}
// Total = 8

pub struct PickaxeBreedingOutcome {
    pub twin_triggered: bool, // 1
    pub boxes_awarded: u8,    // 1
    pub rarity_roll: u16,     // 2
    pub twin_roll: u16,       // 2
}
// Total = 6
```

```rust
pub struct BreedingAttempt {
    // code
    pub outcome: Option<BreedingOutcome>,
    // code
}
```

```rust
const OUTCOME_MAX_SIZE: usize = 1 + PickaxeBreedingOutcome::SERIALIZED_SIZE;
```

This assumes the outcome field needs only: - 1 byte (intended to represent Option<> "some/none"

flag), plus the pickaxe outcome payload size

But the outcome is not Option.

It is: `outcome: Option<BreedingOutcome>` where BreedingOutcome is an enum with variants Gem(...) and Pickaxe(...).

```
/// Breeding outcome summary
#[derive(AnchorSerialize, AnchorDeserialize, Clone, Debug, PartialEq, Eq)
    ]
pub enum BreedingOutcome {
    Gem(GemBreedingOutcome),
    Pickaxe(PickaxeBreedingOutcome),
}
```

So we're missing at least the enum tag byte, and if the "gem outcome" variant is used in the future, it's larger than the pickaxe payload too.

The constant is missing that **enum variant tag byte** at a minimum.

So the maximum `outcome` size should be: - (1) option tag + (1) enum tag + max(payload) - max(payload) = max(8, 6) = 8 - total = **10 bytes**

Current code budgets: - (1 + 6 = 7) bytes

So the attempt account is **under-sized by at least 1 byte even for the Pickaxe variant**, and by **3 bytes** for the Gem variant.

The bug also exists even for the pickaxe path you currently use.

When we store Some(BreedingOutcome::Pickaxe(outcome)), the serialized bytes require: - 1 byte for Option tag (None vs Some) - 1 byte for the enum variant tag (Gem vs Pickaxe) - payload bytes for the pickaxe outcome (6 bytes)

But the current sizing constant only budgets: - 1 byte + pickaxe payload (and misses that enum tag byte)

**However, in the current implementation, this is not exploitable because:** - Pickaxe breeding uses `consumed_gems = 0`, which allocates `max(0, 3) * 4 = 12` bytes for gem capacity - The `consumed_gem_ids` vec is empty and serializes as only 4 bytes (length prefix), leaving 8 bytes unused - The actual serialized size (211 bytes) fits within the total allocated space (222 bytes) - The 1-byte shortfall in `OUTCOME_MAX_SIZE` is covered by the buffer from gem capacity

**Note:** This would become exploitable if gem breeding is implemented with `consumed_gems > 0`, as the buffer would shrink.

### Location of Affected Code

File: game/src/state.rs#L405

```
impl BreedingAttempt {
    pub const MAX_TRACKED_GEMS: usize = 3;
    pub const MAX_AWARDED_BOXES: usize = 2;
    const OUTCOME_MAX_SIZE: usize = 1 + PickaxeBreedingOutcome::
       SERIALIZED_SIZE;   >> @bug

    const BASE_SIZE: usize = 8  // discriminator
        // ...
        + Self::OUTCOME_MAX_SIZE // outcome data (max pickaxe variant)
        // ...
        + 1; // bump

    pub fn space(consumed_gems: usize) -> usize {
        let gem_capacity = core::cmp::max(consumed_gems, Self::
           MAX_TRACKED_GEMS);
        Self::BASE_SIZE + gem_capacity * 4
    }
}
```

### Impact

**Not exploitable in current implementation** – The calculation error is mitigated by buffer space from gem capacity allocation.

**Potential future impact if gem breeding is implemented:** – If gem breeding uses `consumed_gems > 0`, the buffer shrinks and the issue could become exploitable. This would cause breeding resolution failures when attempting to write `Some(BreedingOutcome::Pickaxe(...))` or `Some(BreedingOutcome::Gem(...))`.

### Recommendation

Correct `OUTCOME_MAX_SIZE` to include the Option tag + enum variant tag + max payload for code correctness and future-proofing.

While not exploitable in the current implementation, fixing this ensures: – The calculation accurately reflects the actual serialized size – Future implementations (gem breeding) won't encounter issues – Code maintainability and clarity for future developers

Add a `SERIALIZED_SIZE` constant for `GemBreedingOutcome`, then update `OUTCOME_MAX_SIZE`.

Proposed change:

```
impl GemBreedingOutcome {
    // bool (1) + Option<u32> (1+4) + Option<u8> (1+1)
    pub const SERIALIZED_SIZE: usize = 1 + (1 + 4) + (1 + 1);
}

impl BreedingAttempt {
    // Option tag (1) + enum variant tag (1) + max payload
    const OUTCOME_MAX_SIZE: usize =
        1 + 1 + core::cmp::max(GemBreedingOutcome::SERIALIZED_SIZE,
           PickaxeBreedingOutcome::SERIALIZED_SIZE);

    // BASE_SIZE stays the same but now uses the corrected
       OUTCOME_MAX_SIZE
}
```

That makes `OUTCOME_MAX_SIZE = 10`, which matches the worst case and ensures correctness regardless of buffer space.

### Team Response

Fixed.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

# Thank you!